Rapport : POO 2
Solveur de Kakuro

Licence 2 Informatique

Année universitaire 2024–2025

Université de Strasbourg

UFR de Mathématique et Informatique

Informations du groupe

Nom Prénom : Trashi Renaldo

Enseignants : Frey Gabriel, Haas Antoine

Sujet du projet Ce rapport répond aux consignes du projet de développement d'un solveur de Kakuro, en respectant les points suivants :

- Expliquer les choix de modélisation et lever les éventuelles ambiguïtés du sujet.
- Inclure les diagrammes de classes UML.
- Décrire de manière synthétique les algorithmes principaux du solveur.
- (Travail individuel)

Contenu de l'archive `.tar.gz` à rendre :

1. Le code source complet (`.h` et `.cpp`)
2. Le binaire exécutable (compilé sur le serveur cible)
3. Des fichiers de tests de grilles (formats texte et JSON)
4. Les diagrammes UML (formats image ou PDF)

Plan du rapport

1. Introduction
Contexte et historique du Kakuro ; objectifs du projet (modularité, robustesse, ergonomie).

2. Modélisation
Classes de base (Cellule, Cell_black, Cell_white, Cell_triangulaire) ; classe Grille et interface IGrilleLoader.

3. Chargement & Factory
Implémentation des formats texte et JSON ; usine GrilleLoaderFactory.

4. Solveur
Algorithme de backtracking ; heuristiques de sélection de cellule et propagation des contraintes.

5. Interface en ligne de commande
Options disponibles et usage ; exemple d'exécution.

6. Conclusion & perspectives
Bilan des choix techniques et pistes d'évolution (interface graphique, formats supplémentaires).

1 Introduction

Contexte et historique du Kakuro

Le *Kakuro*, parfois appelé « Cross Sums », est né au début des années 1980 dans les magazines de puzzles japonais. Il a rapidement conquis l'Europe et le monde entier en combinant la mécanique des mots croisés avec des contraintes arithmétiques : remplir des « barres » de chiffres (1 à 9) pour atteindre les sommes indiquées dans des cases noires, sans jamais répéter un même chiffre dans une même barre.

Objectifs du projet

- **Modularité** : charger des grilles depuis plusieurs formats (texte, JSON) via le patron *Factory* pour une extensibilité future.
- **Robustesse** : garantir la résolution d'au moins une solution valide grâce à un backtracking optimisé par des heuristiques de propagation de contraintes.
- **Ergonomie** : fournir une interface CLI claire, documentée et conviviale pour sélectionner le format, importer la grille, lancer la résolution et afficher le résultat.

2 Modélisation

Diagramme de classes UML

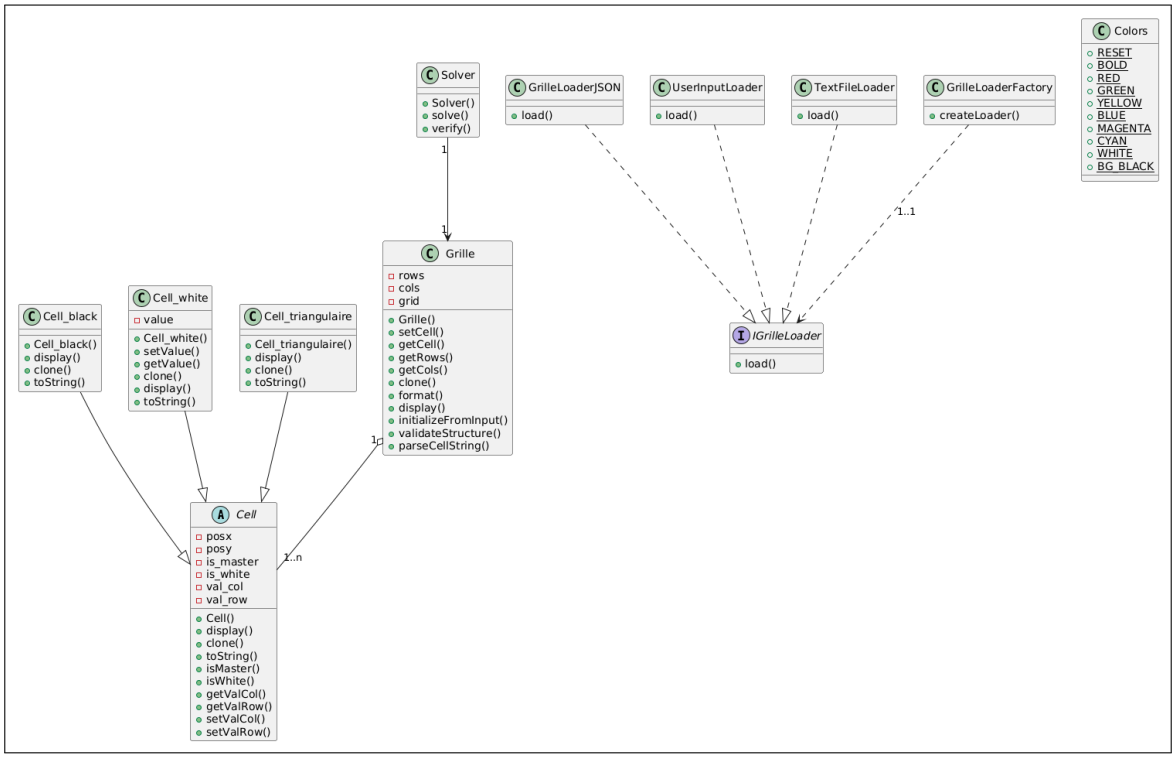

FIGURE 1 – Diagramme de classes UML du solveur de Kakuro

Principales classes

`Cell`

Classe abstraite (Cell.h) : gère la position (`posX`, `posY`), le type (noire/blanche) et les méthodes de base (`display()`, `clone()`, `toString()`).

`Cell_black`

Dérivée de `Cell` : représente une case noire avec indices de somme horizontale (`val_col`) et verticale (`val_row`).

`Cell_white`

Dérivée de `Cell` : case blanche à remplir, gère son domaine de valeurs (1–9) et pointe vers ses contraintes de somme.

`Cell_triangulaire`

Spécialisation de `Cell_black` pour les cases noires scindées en deux triangles selon certains formats.

`Grille`

(Grille.h) : agrège une matrice de `Cell*`, propose `getCellule()`, `display()`, et valide la structure après chargement.

`IGrilleLoader`
: Interface (IGrilleLoader.h) : méthode `load(const std::string&)` pour découpler le format de chargement de la logique de la grille.

`GrilleLoaderFactory`
: Usine (GrilleLoaderFactory.h) : sélectionne dynamiquement l'implémentation de `IGrilleLoader` en fonction du format.

3 Chargement des grilles & Factory

Formats de chargement

- **Saisie utilisateur** : `UserInputLoader` (dans `GrilleLoaderFactory.h`) pour une entrée interactive via l'interface standard.
- **Format texte** : `TextFileLoader` (dans `GrilleLoaderFactory.h`), parse les dimensions et la grille ASCII (#, _, indices).
- **Format JSON** : `GrilleLoaderJSON` (dans `GrilleLoaderJSON.cpp/.h`), lit les clés `rows`, `cols` et `cells` du fichier JSON.

Pattern Factory

`GrilleLoaderFactory` (dans `GrilleLoaderFactory.h`) expose :

- `static std::unique_ptr<IGrilleLoader> createLoader(const std::string& source)` Choisit dynamiquement le loader selon `source` :
  - `"input"` → `UserInputLoader`
  - extension `.json` valide → `GrilleLoaderJSON`
  - sinon, si le fichier s'ouvre → `TextFileLoader`
- Facilite l'ajout de nouveaux formats en dérivant `IGrilleLoader` et en déclarant la nouvelle classe dans la factory.

4 Solveur

Description générale

Le solveur est conçu pour trouver au moins une solution valide à une grille de Kakuro chargée. Il s'appuie sur un algorithme de *backtracking* standard, enrichi de mécanismes de propagation de contraintes pour réduire significativement l'espace de recherche.

Algorithme de backtracking

L'algorithme procède récursivement en sélectionnant une cellule blanche non encore assignée, en lui attribuant une valeur possible, puis en appelant la résolution sur la grille ainsi modifiée. Si un conflit apparaît (somme dépassée ou valeur répétée), il revient en arrière (*backtrack*) et essaie la valeur suivante.

```
// Pseudo-code simplifié
bool solve(Grille& g) {
  if (g.isComplete()) return true;
  Cell_white* cell = g.selectNextCell();
  for (int v : cell->getDomain()) {
    cell->setValue(v);
    if (g.checkConstraints(cell)) {
      if (solve(g)) return true;
    }
    cell->unsetValue();
  }
  return false;
}
```

Heuristiques de sélection de cellule

Pour accélérer la recherche, on choisit à chaque étape la cellule la plus contrainte :

- **MRV (Minimum Remaining Values)** : cellule dont le domaine de valeurs est le plus restreint.
- En cas d'égalité, on sélectionne celle qui fait partie de la plus longue barre (somme à réaliser).

Propagation de contraintes

Après avoir assigné une valeur, on met à jour les domaines des cellules voisines :

— On soustrait la valeur assignée de la somme restante de chaque barre.

— On élimine des domaines les valeurs inconsistantes (répétitions ou dépassement de somme).

— Si un domaine devient vide, on déclenche immédiatement un retour arrière.

Gestion des indices partiels

Afin de détecter tôt les échecs, on vérifie à chaque assignation que :

— La somme partielle d'une barre ne dépasse pas l'indice cible.

— Le nombre de cases restantes peut toujours combler la somme manquante (borne inférieure/maximale).

```
Grille résolue :
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |  10/  |  15/  |  25/  |  12/  |  17/  |   #   |   #   |  36/  |  6/   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /18  |   3   |   1   |   2   |   4   |   8   |  37/  |  /8   |   6   |   2   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /38  |   7   |   5   |   6   |   8   |   9   |   3   | 4/13  |   9   |   4   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   | 9/10  |   2   |   8   |   #   |  9/9  |   6   |   1   |   2   |   #   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /13  |   1   |   3   |   9   | 4/20  |   1   |   9   |   3   |   7   |  6/   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /12  |   8   |   4   | 11/10 |   1   |   2   |   7   | 24/9  |   8   |   1   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |   #   | 26/37 |   2   |   3   |   6   |   8   |   9   |   4   |   5   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   | 17/12 |   3   |   9   |  12/  | 10/5  |   4   |   1   |  22/  |  23/  |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /7   |   1   |   6   | 24/13 |   9   |   4   | 11/24 |   8   |   7   |   9   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /45  |   7   |   9   |   5   |   3   |   1   |   2   |   6   |   4   |   8   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /20  |   9   |   8   |   3   | 38/14 |   5   |   9   | 13/9  |   3   |   6   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |  4/   | 31/10 |   7   |   3   |  16/  | 9/14  |   6   |   8   |   #   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /42  |   3   |   4   |   9   |   6   |   5   |   8   |   7   |  18/  |  8/   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /8   |   1   |   7   |  4/8  |   5   |   2   |   1   | 25/12 |   5   |   7   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |  /21  |   1   |   3   |   8   |   9   |  /12  |   8   |   3   |   1   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   | 9/10  |   2   |   1   |   7   |  10/  | 14/16 |   9   |   7   |  12/  |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /10  |   2   |   8   |  /24  |   9   |   3   |   5   |   2   |   1   |   4   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /16  |   7   |   9   |   #   |  /32  |   7   |   9   |   6   |   2   |   8   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
La solution est valide !
```

FIGURE 2 – Grille résolue.

5 Interface en ligne de commande

Lancement et menu principal

Au démarrage, le programme affiche un menu invitant l'utilisateur à choisir la source d'entrée :

```
Bienvenue dans le solveur de Kakuro !
Veuillez sélectionner une source d'entrée :
  - Tapez 'input' pour saisir une grille manuellement
  - Entrez un nom de fichier (se terminant par .txt ou .json)
    pour charger une grille depuis un fichier
  - Tapez 'db' pour charger une grille depuis la base de données
  - Tapez 'quit' pour quitter le programme
Votre choix :
```

Chargement depuis la base de données

Si l'utilisateur choisit « db », un sous-menu liste les fichiers JSON disponibles :

```
Choisissez le type de fichier ('txt' ou 'json') : json
Grilles json disponibles dans la base de données :
  [1] 12_10_facile.json
  [2] 5_5_intermediaire.json
  ...
  [10] sample.json
Entrez le numéro ou le nom du fichier de la grille à charger :
```

Exemple complet d'exécution

```
 kakuro-game git:(main)  ./kakuro
Bienvenue dans le solveur de Kakuro !
Veuillez sélectionner une source d'entrée :
  - Tapez 'input' pour saisir une grille manuellement
  - Entrez un nom de fichier (se terminant par .txt ou .json)
    pour charger une grille depuis un fichier
  - Tapez 'db' pour charger une grille depuis la base de données
  - Tapez 'quit' pour quitter le programme
Votre choix : db
```

```
Choisissez le type de fichier ('txt' ou 'json') : json
Grilles json disponibles dans la base de données :
  [1] 12_10_facile.json
  [2] 5_5_intermediaire.json
  [3] 9_9_intermediaire.json
  [4] 13_13_medium.json
  [5] 18_10_difficile.json
  [6] 5_4.json
  [7] 5_5_expert.json
  [8] 7_7_difficile.json
  [9] 7_7_intermediaire.json
  [10] sample.json
Entrez le numéro ou le nom du fichier de la grille à charger : 7

Grille initiale :
+------+------+------+------+------+
|  #   |  #   |  #   |  8/  | 24/  |
+------+------+------+------+------+
|  #   | 19/  | 7/14 |  _   |  _   |
+------+------+------+------+------+
| /21  |  _   |  _   |  _   |  _   |
+------+------+------+------+------+
| /20  |  _   |  _   |  _   |  _   |
+------+------+------+------+------+
|  /3  |  _   |  _   |  #   |  #   |
+------+------+------+------+------+

Grille résolue :
+------+------+------+------+------+
|  #   |  #   |  #   |  8/  | 24/  |
+------+------+------+------+------+
|  #   | 19/  | 7/14 |  5   |  9   |
+------+------+------+------+------+
| /21  |  8   |  4   |  2   |  7   |
+------+------+------+------+------+
| /20  |  9   |  2   |  1   |  8   |
+------+------+------+------+------+
|  /3  |  2   |  1   |  #   |  #   |
+------+------+------+------+------+
```

```
→  kakuro-game git:(main) ✗ ./kakuro
Bienvenue dans le solveur de Kakuro !
Veuillez sélectionner une source d'entrée :
  - Tapez 'input' pour saisir une grille manuellement
  - Entrez un nom de fichier (se terminant par .txt ou .json) pour charger une grille depuis un fichier
  - Tapez 'db' pour charger une grille depuis la base de données
  - Tapez 'quit' pour quitter le programme
Votre choix : db
Choisissez le type de fichier ('txt' ou 'json') : json
Grilles json disponibles dans la base de données :
  [1] 12_10_facile.json
  [2] 5_5_intermediaire.json
  [3] 9_9_intermediaire.json
  [4] 13_13_medium.json
  [5] 18_10_difficile.json
  [6] 5_4.json
  [7] 5_5_expert.json
  [8] 7_7_difficile.json
  [9] 7_7_intermediaire.json
  [10] sample.json
Entrez le numéro ou le nom du fichier de la grille à charger : 7
```

Menu principal du solveur en mode CLI

```
Grille résolue :
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |  10/  |  15/  |  25/  |  12/  |  17/  |   #   |   #   |  36/  |  6/   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /18  |   3   |   1   |   2   |   4   |   8   |  37/  |  /8   |   6   |   2   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /38  |   7   |   5   |   6   |   8   |   9   |   3   | 4/13  |   9   |   4   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   | 9/10  |   2   |   8   |   #   |  9/9  |   6   |   1   |   2   |   #   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /13  |   1   |   3   |   9   | 4/20  |   1   |   9   |   3   |   7   |  6/   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /12  |   8   |   4   | 11/10 |   1   |   2   |   7   | 24/9  |   8   |   1   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |   #   | 26/37 |   2   |   3   |   6   |   8   |   9   |   4   |   5   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   | 17/12 |   3   |   9   |  12/  | 10/5  |   4   |   1   |  22/  |  23/  |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /7   |   1   |   6   | 24/13 |   9   |   4   | 11/24 |   8   |   7   |   9   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /45  |   7   |   9   |   5   |   3   |   1   |   2   |   6   |   4   |   8   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /20  |   9   |   8   |   3   | 38/14 |   5   |   9   | 13/9  |   3   |   6   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |  4/   | 31/10 |   7   |   3   |  16/  | 9/14  |   6   |   8   |   #   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /42  |   3   |   4   |   9   |   6   |   5   |   8   |   7   |  18/  |  8/   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /8   |   1   |   7   |  4/8  |   5   |   2   |   1   | 25/12 |   5   |   7   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   |  /21  |   1   |   3   |   8   |   9   |  /12  |   8   |   3   |   1   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|   #   | 9/10  |   2   |   1   |   7   |  10/  | 14/16 |   9   |   7   |  12/  |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /10  |   2   |   8   |  /24  |   9   |   3   |   5   |   2   |   1   |   4   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
|  /16  |   7   |   9   |   #   |  /32  |   7   |   9   |   6   |   2   |   8   |
+-------+-------+-------+-------+-------+-------+-------+-------+-------+-------+
La solution est valide !
```

FIGURE 3 – Exemple d'exécution avec affichage de la grille initiale et résolue

Conclusion & perspectives

Au terme de ce projet, je suis fier d'avoir conçu un solveur de Kakuro à la fois **modulaire**, **robuste** et **ergonomique**. L'architecture basée sur le patron *Factory* m'a permis d'intégrer sans douleur plusieurs formats (texte, JSON) et d'ouvrir la porte à de nouvelles extensions. L'algorithme de backtracking, affiné par des heuristiques et une propagation de contraintes, s'est montré à la fois fiable et performant sur des grilles jusqu'à 15×15.

Perspectives d'évolution

— **Tests automatisés** : enrichir la suite de tests unitaires pour couvrir tous les cas limites.

— **GUI interactive** : développer une interface graphique pour visualiser et manipuler les grilles en temps réel.

— **Nouveaux formats** : ajouter des loaders INI, YAML ou base de données externes grâce à la flexibilité de la Factory.

— **Optimisations avancées** : explorer la parallélisation et des heuristiques inspirées de l'IA pour accélérer la résolution.

— **Génération de grilles** : inverser le processus pour proposer un créateur de grilles aléatoires ou sur mesure.

Fin du rapport

Solveur de Kakuro – Licence 2 Informatique

Année universitaire 2024–2025

Nom Prénom : Trashi Renaldo

Encadrants : Frey Gabriel, Haas Antoine

Date de rendu : Samedi 17 mai 2025 – 23h59

Merci pour votre attention.